Studio Nx

백엔드 312 — 화면 설정 API (GET /client-config)

2026-10-21

백엔드 312 — 화면 설정 API (`GET /client-config`)

항목	내용
파일	`studionx/backend/app/api/v1/router.py`, `studionx/backend/app/schemas/client_config.py`, `studionx/backend/app/core/config.py`, `studionx/.env.example`
작업	S1-13 Sentry 화면 오류 수집: 화면이 시작할 때 읽는 공개 설정(브라우저용 Sentry DSN, 환경, 버전)을 내려 주는 API (PRD 00 §7)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 311 템포 변경점 연결, 다음 313 화면 오류 수집

1. 하는 일

화면(브라우저)이 오류를 Sentry로 보내려면 어느 Sentry 프로젝트로 보낼지(DSN)를 알아야 합니다. DSN을 화면 코드에 적어 두면 환경(개발, 배포)마다 파일을 고쳐야 하므로, 서버가 설정에서 읽어 내려 줍니다.

요청	응답
`GET /api/v1/client-config` (로그인 불필요)	`{ "sentry_dsn": "...", "environment": "dev", "release": "studionx@0.1.0" }`

```
studionx/.env 의 SENTRY_BROWSER_DSN ─▶ Settings.sentry_browser_dsn ─▶ GET /client-config ─▶ 화면(313)
  비어 있으면 화면은 오류를 보내지 않음 (기본값)
```

설정은 둘로 나뉩니다. 서버용 `SENTRY_DSN`은 서버가 낸 Python 오류를 받는 프로젝트이고, 새로 만든 `SENTRY_BROWSER_DSN`은 화면의 JS 오류를 받는 프로젝트입니다.

2. 언제 쓰나

쓰는 곳	하는 일
`core/sentry-init.js` (313)	화면에서 오류가 처음 났을 때 한 번 읽어 DSN을 얻습니다. 오류가 없으면 읽지 않습니다.
시험 `test_client_config.py`	응답 모양, 기본값, 로그인 없이 읽힘, 서버용 DSN이 새지 않음을 확인합니다.

3. 코드

3.1 응답 스키마

```
class ClientConfigResponse(BaseModel):
    """GET /client-config 응답."""

    sentry_dsn: str  # 비어 있으면 화면은 오류를 보내지 않음
    environment: str  # dev | prod (Sentry 의 environment)
    release: str  # 예: studionx@0.1.0
```

3.2 라우터

```
@api_router.get("/client-config", response_model=ClientConfigResponse, tags=["system"])
def client_config(request: Request) -> ClientConfigResponse:
    """화면이 시작할 때 읽는 공개 설정 (S1-13) — 브라우저용 Sentry DSN, 환경, 버전.

    로그인 없이 읽는다. 공개되어도 되는 값만 담는다 (schemas/client_config.py).
    """
    settings = get_settings()
    return ClientConfigResponse(
        sentry_dsn=settings.sentry_browser_dsn,
        environment=settings.app_env,
        release="studionx@" + request.app.version,
    )
```

3.3 설정 칸

```
# core/config.py 의 Settings
    # Sentry — 비어 있으면 비활성화
    sentry_dsn: str = ""                 # 서버(Python) 오류용
    sentry_traces_sample_rate: float = 0.1
    # 브라우저(화면)에서 난 JS 오류를 받는 Sentry 프로젝트의 DSN (S1-13)
    sentry_browser_dsn: str = ""         # 화면(JS) 오류용
```

```
# studionx/.env.example
SENTRY_BROWSER_DSN=
```

4. 왜 이렇게

선택	이유
로그인 없이 읽힙니다	오류는 로그인 화면(홈, 가입)에서도 나고, 로그인이 풀린 상태에서도 납니다. 인증이 필요하면 가장 보고 싶은 오류가 빠집니다.
DSN을 화면에 내려 줘도 됩니다	브라우저용 DSN은 화면 코드에 실려 누구나 볼 수 있게 설계된 공개 값입니다(Sentry 문서도 비밀로 다루지 않음). 이 값으로는 "이 프로젝트에 이벤트를 보내는 것"만 할 수 있고 읽을 수는 없습니다.
서버용 DSN은 절대 담지 않습니다	둘은 다른 값입니다. 시험(`test_server_dsn_never_leaks`)이 서버용 DSN이 응답 어디에도 없음을 확인합니다.
서버용과 브라우저용 프로젝트를 나눕니다	Python 오류와 JS 오류가 한 목록에 섞이면 원인을 가르기 어렵고, 브라우저 오류는 양이 많을 수 있어서 서버 오류가 묻힙니다. 같은 값을 넣어도 동작은 하므로 강제하지는 않습니다.
기본값은 빈 문자열입니다	설정하지 않으면 화면이 아무것도 보내지 않습니다. 개발 중이나 시험 중에 실수로 진짜 Sentry로 가는 일이 없습니다. `tests/conftest.py`가 시험에서는 이 값을 비웁니다.
`release`를 `studionx@0.1.0`으로 만듭니다	앱의 버전(`FastAPI(version=...)`)에서 읽어서 한 곳에서만 올립니다. Sentry가 오류를 버전별로 묶어 보여 줍니다.
화면이 매번 부르지 않고 오류가 날 때 한 번만 부릅니다	대부분의 페이지 로드에서 오류가 없으므로 요청이 늘지 않습니다.

5. 시험

`studionx/backend/tests/test_client_config.py`(4건)가 확인합니다.

- 연결: `/api/v1/client-config`에 `get`만 등록되어 있습니다.
- 기본값: 응답의 키가 `sentry_dsn`, `environment`, `release` 셋이고 DSN이 비어 있으며 `release`가 `studionx@0.1.0`입니다.
- 설정한 값: 브라우저용 DSN과 환경이 응답에 나옵니다. 로그인 헤더 없이도 200입니다.
- 서버용 DSN이 설정되어 있어도 응답 본문에 나타나지 않습니다.
- 백엔드 전체 시험 1109건이 통과합니다 (이전 1105건에 4건 추가). `ruff`도 통과합니다.